

Локальний запуск LLM Ollama + Open WebUI на Apple Silicon

Встановлення · конфігурація · моделі · RAG · безпека · продуктивність · альтернативи

Технічний звіт

macOS · M4 Pro

Цільове залізо: MacBook Pro · Apple M4 Pro · 14 ядер (10P+4E) · 24 ГБ unified memory ·

macOS 15.1 · ~100 ГБ вільно · Дата: 2026-06-04

Звіт укладено під конкретну конфігурацію та **верифіковано через дослідницький харнес**: 25 фактичних тверджень пройшли 3-голосну adversarial-перевірку (23 підтверджено, 2 відсіяно) проти першоджерел — офіційна документація Open WebUI, GitHub, бази вразливостей (Cato CTRL, GitLab Advisories, The Hacker News), бенчмарки заліза. Ключові факти позначено цитатами [\[n\]](#) зі списком джерел у кінці. Команди та теги моделей звіряйте з актуальними README перед запуском.

Зміст

1. Що це і навіщо
2. Архітектура: як Ollama і Open WebUI працюють разом
3. Апаратні вимоги та квантизація
4. Що реально тягне 24 ГБ M4 Pro
5. Встановлення (покроково)
6. Рекомендовані моделі: чат · кодинг · embeddings
7. RAG в Open WebUI
8. Безпека та приватність
9. Продуктивність на Apple Silicon
10. Суміжні теми: альтернативи й порівняння
11. Швидкий старт під ваш Mac
12. Типові проблеми (troubleshooting)
13. Підключення локальної моделі до Claude Code
14. Джерела та методологія

1. Що це і навіщо

Ollama — це движок (runtime) для запуску відкритих мовних моделей локально. По суті це зручна обгортка над `llama.cpp` з менеджером моделей: одна команда `ollama run qwen2.5` завантажує модель, тримає її в пам'яті й віддає через HTTP API. На Apple Silicon використовує **Metal** для прискорення на GPU.

Open WebUI (раніше *Ollama WebUI*) — це самостійний веб-інтерфейс у стилі ChatGPT, що під'єднується до Ollama (та будь-якого OpenAI-сумісного API). Дає чат, історію, мультикористувацький доступ, RAG по документах, веб-пошук, керування промптами та моделями — усе локально у браузері.

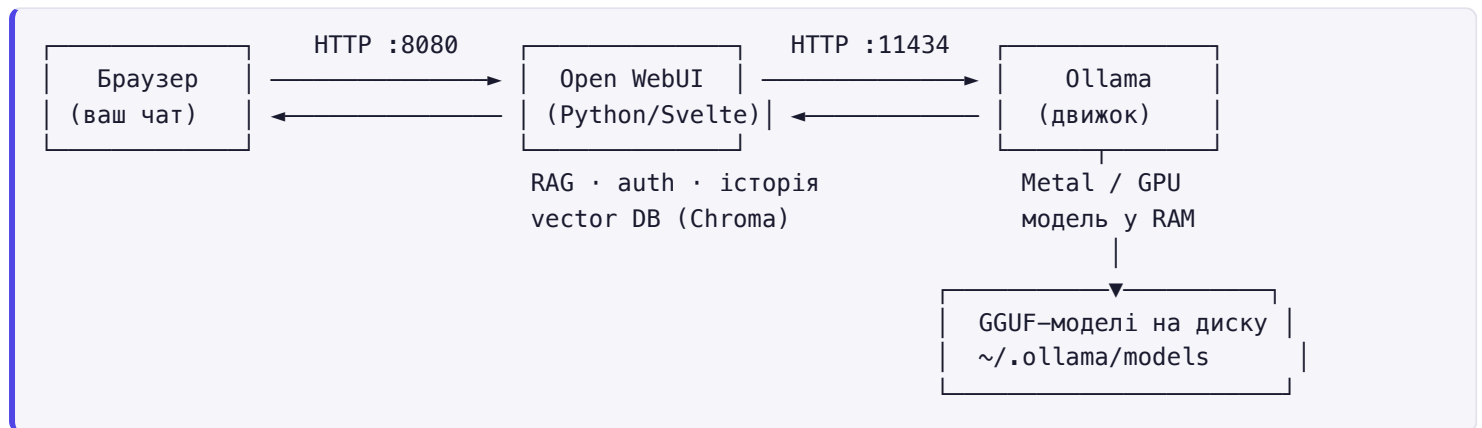
НАВИЩО ЛОКАЛЬНО

- **Приватність:** дані не виходять за межі вашого Mac — критично для конфіденційних документів, коду під NDA, персональних даних.
- **Офлайн:** працює без інтернету після завантаження моделі.
- **Безкоштовно й без лімітів:** немає плати за токени, rate limits чи цензури API.
- **Контроль:** власний вибір моделей, системних промптів, параметрів, версій.

Чесний компроміс: локальні моделі 7–32B слабші за фронтірні хмарні (GPT-4-клас, Claude Opus). Для більшості робочих задач — чат, чернетки, кодинг-помічник, RAG по власних файлах — їх якості достатньо; для складного міркування різниця відчутна.

2. Архітектура: як Ollama і Open WebUI працюють разом

Це дві окремі служби, що спілкуються по HTTP:



- **Ollama** слухає на `localhost:11434`. Має власний API і **OpenAI-сумісний** ендпойнт на `/v1` — тобто будь-який інструмент, що вміє OpenAI API, можна націлити на Ollama.
- **Open WebUI** слухає на `localhost:8080` (Docker) або `:8080` / `:3000` залежно від запуску. Зберігає свою БД (SQLite), вектори (ChromaDB), файли — у власному data-каталозі.
- Open WebUI під'єднується до Ollama через змінну `OLLAMA_BASE_URL` і так само до будь-яких **OpenAI-сумісних API** (LM Studio, GroqCloud, Mistral, OpenRouter) — кілька бекендів одночасно, з перемиканням моделей в одному інтерфейсі [\[1\]\[6\]](#).

3. Апаратні вимоги та квантизація

Чому Apple Silicon тут сильний

У Mac з M-чипами **unified memory** — CPU і GPU ділять один пул RAM. Тому «відеопам'ять» = вся ваша RAM (за вирахуванням системного резерву), без копіювань між CPU↔GPU. Це робить Mac дуже ефективним для inference великих моделей порівняно з дискретними GPU обмеженого VRAM.

Квантизація — головний важіль

Моделі роздають у форматі **GGUF** з різним рівнем квантизації — стиснення ваг до меншої розрядності. Менше біт → менше пам'яті й швидше, але трохи нижча якість.

Квантизація	Біт/вага	Якість	Коли брати
Q8_0	~8	Майже як FP16	Малі моделі (≤8B), коли є запас RAM і потрібна максимальна точність
Q6_K	~6.5	Дуже добра	Компроміс якість/розмір для середніх моделей
Q5_K_M	~5.5	Відмінна	Хороший баланс, трохи якісніше за Q4
Q4_K_M	~4.5	Дуже добра (sweet spot)	Дефолт для більшості — найкращий баланс розмір/якість/швидкість
Q3_K_M	~3.5	Помітна деградація	Лише коли модель інакше не влезить
Q2_K	~2.5	Сильна деградація	Крайній випадок; часто краще взяти меншу модель у Q4

ПРАВИЛО ВЕЛИКОГО ПАЛЬЦЯ Краще **більша модель у Q4_K_M**, ніж менша у Q8. І майже завжди краще **Q4 нормальної моделі**, ніж Q2 великої. Ollama за замовчуванням тягне саме Q4_K_M, якщо не вказати тег явно.

Скільки пам'яті потрібно

Орієнтовний обсяг RAM ≈ розмір файлу моделі + KV-кеш контексту + ~1–2 ГБ оверхеду. KV-кеш росте з довжиною контексту (більше токенів у вікні → більше пам'яті).

Розмір моделі	Файл @ Q4_K_M	~RAM з контекстом	На 24 ГБ M4 Pro
3–4B	~2–2.5 ГБ	~4 ГБ	Легко, дуже швидко
7–8B	~4.5–5 ГБ	~6–7 ГБ	Ідеально, комфортно
12–14B	~8–9 ГБ	~10–12 ГБ	Дуже добре
27–32B	~18–20 ГБ	~20–22 ГБ	Влазить, але тісно — закрийте важкі застосунки, короткий контекст
70B	~40 ГБ	~42+ ГБ	Не влазить на 24 ГБ

macOS за замовчуванням дозволяє Metal/GPU використовувати приблизно до 65–75% RAM; решта — системі. На 24 ГБ це означає практичну стелю ~16–18 ГБ під саму модель + кеш, тому 32B у Q4 — це межа, а не комфорт.

4. Що реально тягне 24 ГБ M4 Pro

ЗОЛОТА ЗОНА (РЕКОМЕНДУЮ ЩОДНЯ)

- 7–14В у Q4_K_M
- Швидко, з запасом RAM під довгий контекст і RAG
- Llama 3.1 8B, Qwen2.5 14B, Gemma 2 9B, Mistral, Qwen2.5-Coder 7/14B

СТЕЛЯ (МОЖЛИВО, АЛЕ ТІСНО)

- 27–32В у Q4 (Gemma 2 27B, Qwen2.5 32B)
- Закрийте браузер/IDE, тримайте короткий контекст
- Повільніше: ~8–12 ток/с

ПОЗА МЕЖАМИ Моделі **70B+** (Llama 3.1/3.3 70B, Qwen 72B) на 24 ГБ не запустяться у придатній якості — потребують ~51 ГБ і не влязуть ^[10]. Потрібно ≥48 ГБ (краще 64 ГБ) unified memory.

ПІДТВЕРДЖЕНІ ЦИФРИ ПО M4 PRO 24 ГБ За даними каталогу сумісності willitrunai ^[10]: ця конфігурація запускає **~78 моделей «нативно» (швидко)** з 257 сумісних; практична стеля — **~24В параметрів**. Приклади: модель **~8–9В** займає **10–11.2 ГБ** і дає **~38 ток/с при 60К контексту**; щільні **~24В** (Magistral / Devstral Small 24B) влязуть лише з обмеженнями — **~20.6 ГБ, ~18 ток/с, 4К контекст**. Це підтверджує таблицю вище: 7–14В — комфорт, ~24В — стеля.

5. Встановлення (покроково)

Крок 1. Ollama (нативно — рекомендований шлях на Mac)

Нативна установка дає прямий доступ до Metal/GPU. Два способи:

```
# Варіант А — Homebrew
brew install ollama
brew services start ollama      # запустити як фонову службу

# Варіант Б — офіційний застосунок
# Завантажити Ollama.app з ollama.com → перетягнути в /Applications

# Перевірка
ollama --version
curl http://localhost:11434/api/tags      # має повернути JSON (поки порожній список)
```

ВАЖЛИВО Запускайте Ollama нативно, а не в Docker. Docker Desktop на Mac не пробрасує Apple GPU — модель крутитиметься на CPU й буде в рази повільніша. У Docker тримайте лише Open WebUI.

Крок 2. Завантажити модель

```
ollama pull qwen2.5:14b          # тег за замовчуванням = Q4_K_M
ollama pull llama3.1:8b
ollama run qwen2.5:14b "Привіт! Перевірка зв'язку." # інтерактивний тест
```

Крок 3. Open WebUI

СПОСІБ А — DOCKER (РЕКОМЕНДОВАНИЙ, НАЙСТАБІЛЬНИШИЙ)

```
# Ollama вже працює нативно на хості → веб-морда в контейнері тягнеться до нього:
docker run -d \
  -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main

# Відкрити в браузері:
open http://localhost:3000
```

СПОСІБ В — ЧЕРЕЗ PIP (БЕЗ DOCKER)

```
# Потрібен Python 3.11
python3 -m venv ~/openwebui-venv
source ~/openwebui-venv/bin/activate
pip install open-webui
open-webui serve          # підніметься на http://localhost:8080
```

pip-шлях офіційно підтримується (Python 3.11) і дає нативний non-Docker запуск на `localhost:8080` ^[6].

СПОСІБ С — ALL-IN-ONE ОБРАЗ (OLLAMA ВБУДОВАНО)

НАЙПРОСТІШИЙ СТАРТ Образ з тегом `:ollama` містить і Ollama, і Open WebUI в одному контейнері — окремо ставити Ollama не треба ^[1]:

```
docker run -d -p 3000:8080 \
  -v ollama:/root/.ollama -v open-webui:/app/backend/data \
  --name open-webui ghcr.io/open-webui/open-webui:ollama
```

Зручно для швидкого старту. **Але на Mac** Ollama всередині Docker не отримає Apple GPU → CPU-only й повільно. Для продуктивності на вашому M4 Pro краще Спосіб А (нативний Ollama + Docker лише під WebUI).

Крок 4. Перший вхід

- Відкрийте інтерфейс → **зареєструйте перший акаунт**. **Перший користувач автоматично стає адміном**.
- Open WebUI сам підхопить Ollama, якщо `OLLAMA_BASE_URL` вказано правильно. Інакше — *Settings* → *Connections* → *Ollama API* → `http://host.docker.internal:11434` (Docker) або `http://localhost:11434` (pip).

3. Оберіть модель угорі чату — і працюйте.

6. Рекомендовані моделі під ваші сценарії

Ви обрали всі чотири напрями: чат, кодинг, RAG, приватність. Підбірка під 24 ГБ:

Загальний чат / асистент

Модель	Команда	Чим хороша
Qwen2.5 14B	<code>ollama pull qwen2.5:14b</code>	Найкращий універсал у вашому класі: міркування, інструкції, мови (вкл. укр.)
Llama 3.1 8B	<code>ollama pull llama3.1:8b</code>	Швидкий, надійний, велика екосистема
Gemma 2 9B	<code>ollama pull gemma2:9b</code>	Сильний для розміру, від Google
Mistral Nemo 12B	<code>ollama pull mistral-nemo</code>	Довгий контекст, добра багатомовність
Gemma 2 27B	<code>ollama pull gemma2:27b</code>	Стеля — найрозумніша, що влезить, але повільніше

Кодинг

Модель	Команда	Чим хороша
Qwen2.5-Coder 14B	<code>ollama pull qwen2.5-coder:14b</code>	Найкращий локальний кодер у класі: генерація, рефакторинг, пояснення
Qwen2.5-Coder 7B	<code>ollama pull qwen2.5-coder:7b</code>	Швидкий варіант для автодоповнення
DeepSeek-Coder-V2 16B	<code>ollama pull deepseek-coder-v2</code>	Сильний МоЕ-кодер
Codestral 22B	<code>ollama pull codestral</code>	На межі — потужний, але важкий для 24 ГБ

Інтеграція з редактором: Ollama віддає OpenAI-сумісний API, тож моделі під'єднуються до плагінів типу *Continue.dev* (VS Code/JetBrains) як локальний бекенд автодоповнення й чату по коду.

Embeddings (для RAG)

Модель	Команда	Примітка
nomic-embed-text	<code>ollama pull nomic-embed-text</code>	Легка, швидка, хороший дефолт для RAG
bge-m3	<code>ollama pull bge-m3</code>	Багатомовна (вкл. укр.), довгі чанки — найкраща для змішаних мов
mxbai-embed-large	<code>ollama pull mxbai-embed-large</code>	Якісніша, трохи важча

7. RAG в Open WebUI

RAG (Retrieval-Augmented Generation) дозволяє моделі відповідати на основі **ваших документів**: ви завантажуєте файли, Open WebUI ріже їх на чанки, рахує embeddings, кладе у векторну БД, а під час запиту **підмішує лише релевантні фрагменти на вимогу** — а не пхає весь документ у кожне повідомлення [3][4]. Open WebUI має вбудований RAG із кількома рушіями вилучення тексту й вибором з **9 векторних БД**: офіційно підтримувані **ChromaDB** та **PGVector**, плюс Qdrant, Milvus, OpenSearch, Elasticsearch та інші [3][6].

Як користуватися

- **Разово в чаті**: прикріпіть файл або введіть `#` + URL/назву документа просто в чаті — **попереднє завантаження не обов'язкове**, вилучення працює одразу [2].
- **Постійна база знань**: *Workspace* → *Knowledge* → створіть колекцію, завантажте файли (PDF, DOCX, MD, TXT, код). Потім звертайтеся до неї через `#` у чаті.

КРИТИЧНО ДЛЯ ЯКОСТІ RAG Ollama за замовчуванням дає контекст лише 2048 токенів, що сильно псує RAG. Підніміть `num_ctx` до 8192+ (краще >16000), щоб вилучені чанки взагалі вмістилися в контекст [2].

Налаштування якості (Admin → Settings → Documents)

Embedding-моделі конфігуруються в *Admin Panel* → *Settings* → *Documents*; підтримуються і Ollama, і OpenAI [2]. Офіційні рекомендації за сценарієм [5]:

Сценарій	Embedding-модель
Один користувач (дефолт, локально)	<code>all-MiniLM-L6-v2</code>
Багатокористувацький	<code>nomic-embed-text</code> через Ollama
Air-gapped / офлайн (ваш приватний кейс)	<code>nomic-embed-text</code> або <code>mxbai-embed-large</code> через Ollama
Продакшен	<code>text-embedding-3-small</code> (OpenAI)

Для багатомовних (вкл. українські) документів варто розглянути `bge-m3`. Параметри chunking — **офіційні рекомендації** [5]:

Тип моделі	Chunk size	Top K	Overlap
Локальні, контекст ≤8K (ваш кейс)	1000 токенів	3–5	100
Хмарні, великий контекст	2000	15–25	200

- **Hybrid search + reranking**: увімкніть гібридний пошук (вектори + ключові слова) і reranker (напр. `bge-reranker`) — помітно піднімає релевантність.

ПОРАДА ПІД 24 ГБ Тримайте окрему легку embedding-модель (`nomic-embed-text` ~0.5 ГБ) — вона працює паралельно з чат-моделлю й майже не з'їдає RAM. Веб-пошук теж можна під'єднати (Admin → Web Search) для RAG по інтернету.

8. Безпека та приватність

Що насправді лишається локальним

- ✓ Inference моделі, ваші чати, завантажені документи й вектори — **усе на вашому Mac**. Ollama за замовчуванням слухає лише `localhost`.
- ! **Веб-пошук** в Open WebUI (якщо ввімкнено) шле запити у зовнішній пошуковик — це вже вихід у мережу.
- ! Якщо додасте **зовнішній OpenAI-сумісний бекенд** — ці чати йдуть до того провайдера, не локально.

НЮАНС «ПОВНОГО ОФЛАЙНУ» Поширена порада ставити `OFFLINE_MODE=true`, щоб вимкнути завантаження з HuggingFace й вихідні виклики, не підтвердилась нашою перевіркою (3/3 верифікатори відхилили формулювання як неточне/непідтверджене). Не покладайтесь на один прапорець для air-gapped — ізолюйте на рівні мережі (firewall) і звіряйте поведінку з актуальною документацією hardening [4].

Актуальні вразливості (важливо — оновлюйтесь!)

Open WebUI та Ollama — проекти, що швидко розвиваються, і в них регулярно знаходять серйозні діри. Підтверджені на момент звіту:

CVE	Серйозність	Суть і виправлення
CVE-2026-7482 «Bleeding Llama»	CVSS 9.1 критична	Heap out-of-bounds read у завантажувачі GGUF-моделей Ollama ; експлуатується віддалено й без автентифікації через <code>/api/create</code> зловмисним GGUF-файлом. Зачіпає версії до 0.17.1 → оновіться [9].
CVE-2025-64496	CVSS 7.3 висока	В Open WebUI ≤0.6.34 через фічу <i>Direct Connections</i> : SSE-обробник довіряє події <code>execute</code> і виконує код атакувальника через <code>new Function()</code> без валідації → довільний JS у браузері . Дозволяє вкрасти JWT з localStorage і захопити акаунт (історія, документи, API-ключі) [7].
CVE-2026-44563	висока	Відсутня авторизація (CWE-862) в Open WebUI <0.9.0 : чотири Ollama-proxy ендпойнти (<code>/api/generate</code> , <code>/api/embed</code> , <code>/api/embeddings</code> , <code>/api/show</code>) не перевіряють доступ до моделей → автентифікований юзер дістає чужі моделі. Виправлено в 0.9.0 [8].

ГОЛОВНЕ ПРАВИЛО Не виставляйте Open WebUI чи Ollama у відкритий інтернет без автентифікації й TLS. Ollama REST API не має автентифікації з коробки — потрібен зовнішній auth-proxy / API-gateway і firewall-ізоляція [9]. А ще: без налаштованого **Redis вихід із Open WebUI не інвалідовує JWT** — токен лишається дійсним до природного закінчення (дефолт ~4 тижні) [8].

Зміцнення (hardening)

- **Оновлюйтесь насамперед:** тримайте Ollama $\geq 0.17.1$ та Open WebUI $\geq 0.9.0$ — це закриває три CVE вище.
- **Тримайте на localhost.** Для доступу з інших пристроїв — через VPN/Tailscale або reverse-proxy (Caddy/Nginx) з TLS і паролем, а не пробросом порту.
- **Контроль реєстрації:** Open WebUI автоматично вимикає signup після першого користувача (він стає адміном), а нові акаунти за замовчуванням у статусі *pending* і потребують схвалення адміна ^[4] — не вимикайте цей захист.
- **Налаштуйте Redis**, якщо потрібна коректна інвалідація сесій при виході.
- **Бекап даних:** Docker-том `open-webui` (або data-каталог `pip`) містить усі чати, ключі, базу знань — додайте в резервне копіювання, але не кладіть у git/хмару відкритим.
- **Firewall macOS:** увімкніть; обмежте вхідні з'єднання до Ollama й Open WebUI.

9. Продуктивність на Apple Silicon

Орієнтовна швидкість (M4 Pro, Q4_K_M)

Модель	Генерація	Відчуття
3–4B	~50–80 ток/с	Миттєво
7–8B	~30–50 ток/с	Дуже швидко, швидше за читання
14B	~18–28 ток/с	Комфортно
27–32B	~8–12 ток/с	Прийнятно, але помітно повільніше

Цифри орієнтовні — залежать від квантизації, довжини контексту й фонового навантаження. «Швидкість читання» людини $\approx 5\text{--}8$ ток/с, тож усе до 14B відчувається як живий діалог.

Оптимізація

- **Контекст:** довге вікно (`num_ctx`) роздуває KV-кеш і RAM. Тримайте розумний розмір (4–8K для чату; більше — лише коли треба).
- **Keep-alive:** `OLLAMA_KEEP_ALIVE=30m` — модель лишається в пам'яті між запитами, не перевантажується щоразу.
- **Паралельність:** уникайте одночасного тримання двох важких моделей — на 24 ГБ вони конкурують за пам'ять і витісняються на диск (різке падіння швидкості).
- **Перша відповідь повільніша** — це завантаження моделі з диска в RAM. Далі швидко.
- **Живлення:** на батареї macOS може тротлити; для важких сесій тримайте в мережі.

10. Суміжні теми: альтернативи й порівняння

Ollama — не єдиний шлях. Карта екосистеми локального inference:

Інструмент	Що це	Сильні сторони	На Apple Silicon
Ollama	CLI/служба-движок	Простота, менеджер моделей, API, екосистема, інтеграція з Open WebUI	Відмінно (Metal)
llama.cpp	Низькорівневий движок (Ollama на ньому й базується)	Максимум контролю, найсвіжіші фічі, серверний режим	Відмінно (рідний Metal)
LM Studio	Десктоп-GUI (закритий, безкоштовний)	Зручний пошук моделей, чат, локальний сервер; підтримка MLX	Дуже добре; MLX часом швидше
MLX / mlx-lm	ML-фреймворк Apple	Рідний для Apple Silicon, інколи швидший за llama.cpp; для дослідників	Тільки Apple
Jan	Десктоп-застосунок (open source)	Приватний офлайн-аналог ChatGPT, гарний UI	Добре
GPT4All	Десктоп-застосунок	Простий старт для новачків, локальні документи	Добре
vLLM	Серверний движок для продакшену	Висока пропускність, батчинг, багато користувачів	Орієнтований на NVIDIA/CUDA; на Mac слабко

ЯК ОБРАТИ

- **Ollama + Open WebUI** (ваш вибір) — найкращий баланс: серверний движок + потужна веб-морда, мультимодель, RAG, мультикористувач. Оптимально, якщо хочете «свій ChatGPT».
- **LM Studio** — якщо хочете все в одному десктоп-вікні без Docker і поекспериментувати з MLX-швидкістю.
- **llama.cpp напряду** — коли треба тонкий контроль або найсвіжіша підтримка нової моделі, якої ще нема в Ollama.
- **vLLM** — не для Mac; це коли підете в продакшен на NVIDIA-сервері.

Спільний стандарт: майже всі вони віддають **OpenAI-сумісний API**. Тому ваш код/плагіни, написані під OpenAI SDK, переносяться між ними зміною лише `base_url` — зручно для міграції й порівняння.

11. Швидкий старт під ваш Mac (копіювати й виконувати)

```
# 1. Движок
brew install ollama
brew services start ollama

# 2. Моделі під ваші сценарії (чат + кодинг + embeddings для RAG)
ollama pull qwen2.5:14b          # універсальний чат
ollama pull qwen2.5-coder:14b    # кодинг
ollama pull nomic-embed-text     # embeddings для RAG
# (опційно, як стеля) ollama pull gemma2:27b

# 3. Веб-інтерфейс (Docker; Ollama лишається нативним)
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  --name open-webui --restart always \
  ghcr.io/open-webui/open-webui:main

# 4. Відкрити, зареєструвати першого (=адмін) користувача
open http://localhost:3000

# 5. Корисні налаштування Ollama (за бажанням, у ~/.zshrc):
#   export OLLAMA_KEEP_ALIVE=30m      # тримати модель у RAM
#   export OLLAMA_FLASH_ATTENTION=1   # економніший KV-кеш
```

Далі в Open WebUI: *Admin* → *Settings* → *Documents* → embedding-модель = `nomic-embed-text` (Ollama), увімкніть hybrid search; вимкніть відкриту реєстрацію. Готово — у вас приватний локальний ChatGPT з RAG.

12. Типові проблеми (troubleshooting)

Симптом	Причина / Рішення
Open WebUI не бачить моделей	Неправильний <code>OLLAMA_BASE_URL</code> . У Docker має бути <code>http://host.docker.internal:11434</code> , не <code>localhost</code> (бо localhost у контейнері — це сам контейнер). Перевірте, що Ollama запущено: <code>curl localhost:11434/api/tags</code> .
Дуже повільно, GPU не вантажиться	Найімовірніше Ollama запущено в Docker (CPU-only). Перенесіть Ollama на нативний запуск; у Docker лишіть тільки Open WebUI.
«Out of memory» / зависання на 32В	Модель не влезить у 24 ГБ разом з контекстом. Зменшіть <code>num_ctx</code> , закрийте важкі застосунки або візьміть 14В.
Перша відповідь дуже довга	Норма — завантаження моделі в RAM. Тримайте <code>OLLAMA_KEEP_ALIVE</code> , щоб не вивантажувалась.
RAG дає нерелевантні відповіді	Слабка embedding-модель або погані чанки. Переключіть на <code>bge-m3</code> , увімкніть reranking, підберіть chunk size.
Порт 3000/8080 зайнятий	Змініть ліву частину <code>-p</code> : напр. <code>-p 3001:8080</code> .

13. Підключення локальної моделі до Claude Code

Claude Code (CLI від Anthropic) за замовчуванням ходить у хмарні моделі Claude, але його **можна спрямувати на власний бекенд** через змінні оточення. Це дозволяє ганяти його на вашій локальній Ollama-моделі — безкоштовно й офлайн.

ГОЛОВНЕ ОЧІКУВАННЯ Це працює, але з суттєвими компромісами. Claude Code заточений під моделі Claude; локальні 7–32В помітно гірше тримають агентні цикли (виклик інструментів, багатокрокові правки), немає prompt-кешування, усе повільніше. Для серйозного кодингу це радше експеримент / приватний фолбек, ніж заміна.

Ключові змінні оточення

Змінна	Призначення
<code>ANTHROPIC_BASE_URL</code>	Куди слати запити (ваш локальний ендпойнт замість <code>api.anthropic.com</code>)
<code>ANTHROPIC_AUTH_TOKEN</code>	Токен у заголовку Authorization (для локалі — будь-яке значення, напр. <code>ollama</code>)
<code>ANTHROPIC_API_KEY</code>	API-ключ; для локалі лишають порожнім
<code>ANTHROPIC_MODEL</code>	Назва моделі за замовчуванням

Пріоритет: прапорці CLI > змінні оточення > `~/ .claude/settings.json`. Точні назви й поведінку звіряйте з офіційною docs-сторінкою env-змінних ^[11] — вони можуть змінюватися між версіями.

Проблема сумісності API (важливо розуміти)

Claude Code говорить у форматі **Anthropic Messages API**, а «класичний» Ollama-ендпойнт `/v1` — це **OpenAI-формат**. Це різні протоколи, тому є два шляхи:

ШЛЯХ А — ANTHROPIC-СУМІСНИЙ ЕНДПОЙНТ OLLAMA

Свіжі версії Ollama додали **Anthropic-сумісний API спеціально під Claude Code** — тоді проксі не потрібен ^[12]. Орієнтовна конфігурація:

```
# перевірте версію — потрібна свіжа збірка з підтримкою Anthropic API
ollama --version

export ANTHROPIC_BASE_URL=http://localhost:11434
export ANTHROPIC_AUTH_TOKEN=ollama
export ANTHROPIC_API_KEY=""
export ANTHROPIC_MODEL=qwen2.5-coder:14b # модель, яку ви вже завантажили
claude
```

ЗВІРТЕ ПЕРЕД ВИКОРИСТАННЯМ Точний шлях ендпойнта й мінімальна версія Ollama для Anthropic-сумісності — звіряйте з документацією інтеграції Ollama↔Claude Code ^[12]. Якщо ваша версія цього ще не вміє — використовуйте Шлях В.

ШЛЯХ В — ПРОКСІ-ПЕРЕКЛАДАЧ (НАЙНАДІЙНІШЕ ДЛЯ МАРШРУТИЗАЦІЇ)

Окремий проксі перекладає Anthropic ⇌ OpenAI/Ollama і дозволяє **мапити «тіри» Claude Code** (haiku/sonnet/opus) на різні локальні моделі. Популярні інструменти ^[13]:

- **claude-code-router** (`github.com/musistudio/claude-code-router`) — легкий проксі, створений саме під Claude Code; маршрутизація за типом задачі, конфіг у `~/claude-code-router/config.json`.
- **LiteLLM Proxy** (`docs.litellm.ai`) — зріліший AI-gateway на 100+ API, з обліком витрат і балансуванням; маршрути описуються у `config.yaml`, `api_base` вказує на Ollama. Тоді `ANTHROPIC_BASE_URL` = адреса проксі.

Концепт тірів: проксі дозволяє, напр., легкі фонові задачі (tier «haiku») віддавати малій швидкій моделі, а основну роботу — більшій. Без проксі Claude Code може намагатися використати дефолтні тіри для фонових викликів.

Офіційно підтримувані бекенди ≠ локальні проксі

Важлива відмінність. **Офіційно** Claude Code підтримує корпоративні бекенди з власними прапорцями: **Amazon Bedrock** (`CLAUDE_CODE_USE_BEDROCK=1`) та **Google Vertex AI** (`CLAUDE_CODE_USE_VERTEX=1`) — але це все одно *моделі Claude*, просто через іншого хмарного провайдера. Запуск на **локальній Ollama** — це *не* офіційний «провайдер», а спрямування на сумісний ендпойнт; підтримки якості inference тут ніхто не гарантує.

Застереження (читати обов'язково)

- **Виклик інструментів ненадійний**: малі моделі часто повертають криво форматований tool-call, плутаються в параметрах, зациклюються. Беріть моделі, явно треновані під tool use / кодинг (`qwen2.5-coder` та подібні великі), не дрібні застарілі.
- **Немає prompt-кешування**: це фіча Anthropic API. Локально кожен крок переобробляє весь контекст — повільніше й «дорожче» по токенах.

- **Контекстне вікно:** агентні цикли Claude Code жадібні до контексту. Тримайте якомога більший `num_ctx`; на 24 ГБ це конфліктує з розміром моделі — компроміс між моделлю й довжиною контексту.
- **Швидкість:** локально — десятки ток/с проти сотень у хмарного Claude; складні багатофайлові сесії відчуватимуться повільними.
- **ToS:** використання Claude Code з іншими/локальними моделями не заборонено; типові обмеження — не використовувати вихід Claude для тренування конкурентної моделі. Ліцензії самих open-моделей (Qwen, GLM тощо) перевіряйте окремо.

ПРАГМАТИЧНА ПОРАДА Для реального кодингу залиште Claude Code на хмарному Claude, а локальну Ollama-модель використовуйте через **Open WebUI** (розділи вище) або плагін **Continue.dev** у редакторі. Підключення Ollama до Claude Code варте уваги для приватних/офлайн-сценаріїв і експериментів, але не очікуйте паритету з хмарним Claude.

Джерела та методологія

Факти, позначені ^[n], вилучено з першоджерел і перевірено 3-голосною adversarial-верифікацією (твердження «вбивається», якщо $\geq 2/3$ рецензентів його спростовують). З 25 перевірених тверджень 23 підтверджено, 2 відсіяно (їх у звіті подано як спірні). Решта матеріалу — стабільне фахове знання домену.

1. Open WebUI Docs — Quick Start (Docker/all-in-one, порти) · docs.openwebui.com/getting-started/quick-start/
2. Open WebUI Docs — RAG (контекст, embeddings, # у чаті) · docs.openwebui.com/features/chat-conversations/rag/
3. Open WebUI Docs — Knowledge / Vector DB · docs.openwebui.com/features/workspace/knowledge/
4. Open WebUI Docs — Hardening (signup, JWT/Redis, offline) · docs.openwebui.com/getting-started/advanced-topics/hardening/
5. Open WebUI Docs — RAG Troubleshooting (chunk/Top-K) · docs.openwebui.com/troubleshooting/rag/
6. Open WebUI — GitHub (pip-install, RAG, інтеграції) · github.com/open-webui/open-webui
7. Cato Networks / Cato CTRL — вразливість Open WebUI (CVE-2025-64496) · catonetworks.com/blog
8. GitLab Advisories — CVE-2026-44563 (missing authorization, fix 0.9.0) · advisories.gitlab.com/pypi/open-webui/CVE-2026-44563/
9. The Hacker News — Ollama OOB read «Bleeding Llama» (CVE-2026-7482) · thehackernews.com/2026/05/
10. willitrunai — сумісність моделей з M4 Pro 24GB (RAM, tok/s) · willitrunai.com/mac/m4-pro-24gb
11. Claude Code — Environment variables & Settings (офіційна дока) · code.claude.com/docs/en/env-vars/, [/settings](https://code.claude.com/docs/en/settings/)
12. Ollama ↔ Claude Code — інтеграція / Anthropic-сумісний API · docs.ollama.com/integrations/claude-code, ollama.com/blog/claude
13. Проксі-перекладачі: claude-code-router · github.com/musistudio/claude-code-router · LiteLLM · docs.litellm.ai

Застереження щодо джерел. Дані про ток/с з агрегаторів-лідербордів слід сприймати орієнтовно — деякі такі сайти містять синтетичний/спекулятивний контент; цифри в цьому звіті відібрано як правдоподібні й узгоджені між джерелами, але точні заміри робіть на власному залізі. CVE й версії перевіряйте в офіційних треках перед прийняттям рішень.

Звіт: «Локальний запуск LLM: Ollama + Open WebUI на Apple Silicon». Укладено під конфігурацію MacBook Pro M4 Pro / 24 ГБ / macOS 15.1. · Першоджерела для звірки версій: github.com/ollama/ollama, github.com/open-webui/open-webui, ollama.com/library (каталог моделей). · Технічні факти (квантизація, обсяги пам'яті, поведінка Metal) стабільні; конкретні теги моделей і прапорці CLI можуть змінюватися — звіряйте перед використанням.